Django E-Commerce

Full-Stack Web Application

Django 2.2 | Stripe | django-allauth | Bootstrap / MDB

1. Project Overview

This is a full-stack Django e-commerce web application that allows users to browse products, manage a shopping cart, apply coupon codes, and complete purchases through Stripe payment processing. It also provides a full order lifecycle — including fulfilment status tracking and refund requests — all managed through Django's built-in admin interface.

1.1 Core Features

- Product catalogue with category, label, discount pricing, and slug-based product detail pages.
- Shopping cart: add items, adjust quantity (increment / decrement / full remove), and view an order summary.
- Multi-step checkout: separate shipping and billing address collection with default-address recall per user.
- Coupon code redemption that deducts a fixed amount from the order total at checkout.
- Stripe payment processing: one-off token charges and saved card / one-click purchasing via Stripe Customer objects.
- Full order lifecycle flags: ordered, being_delivered, received, refund_requested, refund_granted.
- Refund request form — customers submit a ref_code, reason, and email; admins approve via the Django admin.
- User authentication via django-allauth: registration, login, logout, email verification, and password reset.
- Custom UserProfile with Stripe customer ID storage and one_click_purchasing flag.
- Django admin customisation: rich OrderAdmin with list filters, search, and a bulk 'mark refund granted' action.

1.2 Technology Stack

Layer	Technology	Version / Notes
Web Framework	Django	2.2.14
Language	Python	3.x
Database (dev)	SQLite	db.sqlite3 — zero-config for development
Database (prod)	PostgreSQL	Via psycopg2; configured in production.py
Payment Processing	Stripe	stripe==2.27.0 — Charge API + Customer API
Authentication	django-allauth	0.39.1 — login, signup, email verification, social auth ready
Forms	django-crispy-forms	1.7.2 — Bootstrap 4 form rendering
Country Fields	django-countries	5.3.3 — CountryField + CountrySelectWidget

Config Management	python-decouple	3.1 — .env file via config()
Image Handling	Pillow	6.2.2 — ImageField support for product images
Frontend CSS/JS	Bootstrap + MDB	Bootstrap 4 + Material Design for Bootstrap
Dev Tooling	django-debug-toolbar	1.10.1 — SQL, template, cache inspection in dev

2. System Architecture

2.1 Project Layout

The project follows Django's standard structure with one primary Django app (core) housing all business logic, and a project-level package (djecommerce) containing settings, root URL configuration, and WSGI. Settings are split into three files — base, development, and production — allowing environment-specific overrides with no code duplication.

Path	Purpose
djecommerce/settings/base.py	Shared settings: installed apps, middleware, templates, static/media paths, allauth config
djecommerce/settings/development.py	Dev overrides: DEBUG=True, SQLite DB, Stripe test keys, debug-toolbar setup
djecommerce/settings/production.py	Prod overrides: DEBUG from .env, PostgreSQL DB, Stripe live keys, password validators
djecommerce/urls.py	Root URL conf: admin/, accounts/ (allauth), and core app URLs; debug-toolbar in dev
core/models.py	All data models: Item, OrderItem, Order, Address, Payment, Coupon, Refund, UserProfile
core/views.py	All views: HomeView, ItemDetailView, cart functions, CheckoutView, PaymentView, refund views
core/urls.py	App-namespaced URL patterns for all core views
core/forms.py	CheckoutForm, CouponForm, RefundForm, PaymentForm
core/admin.py	Custom admin classes for Order and Address; bulk refund action
core/templatetags/cart_template_tags.py	cart_item_count template filter — shows live cart count in navbar
templates/	All Blade-style Django templates: base.html, home.html, product.html, checkout.html, payment.html, etc.
static_in_env/	Frontend assets: Bootstrap, MDB CSS/JS, jQuery, custom SCSS, fonts, images
.env	Environment variables: SECRET_KEY, DEBUG, DB credentials, Stripe keys (gitignored in real deployments)

2.2 Settings Architecture

The settings split uses Python module inheritance: development.py and production.py both begin with 'from .base import *' and then override only what differs. The active settings module is specified via the DJANGO_SETTINGS_MODULE environment variable. Sensitive values (SECRET_KEY, database credentials, Stripe keys) are loaded with python-decouple's config() function from the .env file, keeping secrets out of source code.

Setting	Development	Production
DEBUG	True	From .env (False in production)
Database	SQLite (db.sqlite3)	PostgreSQL via psycopg2
Stripe Keys	STRIPE_TEST_*	STRIPE_LIVE_*
Debug Toolbar	Enabled	Not installed
Password Validators	Not enforced	4 validators enforced
ALLOWED_HOSTS	['127.0.0.1']	IP address + domain name

3. Data Models

All models live in core/models.py and are backed by Django's ORM. The schema supports a complete purchase lifecycle from browsing to fulfilment and refunds.

3.1 Model Reference

Model	Key Fields	Purpose
UserProfile	user (OneToOne), stripe_customer_id, one_click_purchasing	Extends Django's User; stores Stripe Customer ID and one-click card preference. Auto-created via post_save signal on User creation.
Item	title, price, discount_price, category, label, slug, description, image	A product in the catalogue. Category choices: Shirt / Sport wear / Outwear. Label choices drive Bootstrap badge colour (primary/secondary/danger).
OrderItem	user (FK), item (FK), quantity, ordered	A line item — one product + quantity in an order. Provides price helpers: get_total_item_price(), get_final_price() (uses discount_price if present), get_amount_saved().
Order	user (FK), ref_code, items (M2M OrderItem), ordered, shipping_address (FK), billing_address (FK), payment (FK), coupon (FK), being_delivered, received, refund_requested, refund_granted	The shopping cart / order. ordered=False = active cart. get_total() sums OrderItem final prices and deducts coupon amount.
Address	user (FK), street_address, apartment_address, country (CountryField), zip, address_type (B/S), default	Reusable shipping or billing address per user. default=True addresses are pre-filled in checkout.
Payment	stripe_charge_id, user (FK), amount, timestamp	Records a completed Stripe charge. Linked to the Order after successful payment.
Coupon	code, amount	A promo code that deducts a fixed dollar amount from the order total.
Refund	order (FK), reason, accepted, email	A customer refund request. accepted is toggled by admin via the bulk make_refund_accepted action.

3.2 Key Relationships

- User (1) ——< Order (many): A user can have multiple orders; only one with ordered=False at a time (the active cart).

- Order (M2M) ——< OrderItem: An order holds multiple line items; each OrderItem belongs to one Order.
- OrderItem (many) ——> Item (1): Each line item references one product; the same product can appear in many orders.
- Order (many) ——> Address (1): Separate FK for shipping_address and billing_address, both nullable — set during checkout.
- Order (many) ——> Payment (1): Linked after Stripe charge succeeds; null until then.
- Order (many) ——> Coupon (1): Optional coupon applied at checkout.
- User (1) ——> UserProfile (1): Created automatically via post_save signal the moment a User is created.

3.3 Order Lifecycle

Stage	ordered	being_delivered	received	refund_*
Active cart	False	False	False	False / False
Paid & placed	True	False	False	False / False
Shipped	True	True	False	False / False
Delivered	True	True	True	False / False
Refund requested	True	True	True	True / False
Refund approved	True	True	True	False / True

4. Views & URL Routing

4.1 URL Reference

URL Pattern	View	Auth	Description
/ (home)	HomeView (ListView)	No	Paginated product listing (10 per page)
/product/<slug>/	ItemDetailView (DetailView)	No	Single product detail page
/add-to-cart/<slug>/	add_to_cart	Yes	Adds item to cart or increments quantity; creates Order if none active
/remove-from-cart/<slug>/	remove_from_cart	Yes	Completely removes an item from the active order
/remove-item-from-cart/<slug>/	remove_single_item_from_cart	Yes	Decrements quantity by 1; removes if quantity reaches 0
/order-summary/	OrderSummaryView	Yes	Displays active cart contents and totals
/checkout/	CheckoutView	Yes	GET: checkout form with saved address recall; POST: save addresses, route to payment
/add-coupon/	AddCouponView	Yes	Validates and applies a coupon code to the active order
/payment/<payment_option>/	PaymentView	Yes	GET: payment page with Stripe public key; POST: create Stripe charge, finalise order
/request-refund/	RequestRefundView	No	GET: refund form; POST: create Refund object, set order.refund_requested=True
/accounts/*	django-allauth	—	Login, signup, logout, email confirm, password reset, social auth
/admin/	Django Admin	Staff	Full model admin with custom OrderAdmin and AddressAdmin

4.2 View Breakdown

HomeView

- Django generic ListView on Item model. Paginated at 10 items. Template: home.html.

ItemDetailView

- Django generic DetailView on Item model. Looks up by slug. Template: product.html.

add_to_cart (function-based, @login_required)

- get_or_create an OrderItem for the current user + item + ordered=False.
- If an active Order exists and already contains this item: increment quantity.
- If an active Order exists but does not contain this item: add the OrderItem to the order.
- If no active Order exists: create a new Order, add the item.

remove_from_cart / remove_single_item_from_cart

- remove_from_cart: removes the OrderItem entirely from the order and deletes the record.
- remove_single_item_from_cart: decrements quantity by 1; if quantity reaches 1 and is decremented it removes the item entirely.

CheckoutView (class-based View)

- GET: retrieves active order, pre-populates form with any saved default shipping/billing addresses, renders checkout.html.
- POST: processes four address scenarios — use_default_shipping, new shipping address, same_billing_address, use_default_billing, new billing address. Each scenario saves or reuses Address records. Routes to /payment/stripe/ or /payment/paypal/ based on payment_option selection.

PaymentView (class-based View)

- GET: retrieves the active order; if one_click_purchasing is enabled fetches saved cards from Stripe Customer; renders payment.html with Stripe public key.
- POST: handles three payment paths — (1) save new card: creates or retrieves Stripe Customer, adds card source, charges Customer; (2) use_default: charges existing Stripe Customer; (3) one-off: charges the raw Stripe token directly.
- On success: creates a Payment record, marks all OrderItems as ordered=True, sets order.ordered=True, generates a 20-character alphanumeric ref_code, links the Payment.
- All six Stripe exception types are caught individually with user-facing flash messages: CardError, RateLimitError, InvalidRequestError, AuthenticationError, APIConnectionError, StripeError.

AddCouponView

- Validates the CouponForm, looks up the Coupon by code, attaches it to the active Order. Coupon is deducted in Order.get_total().

RequestRefundView

- Customer submits ref_code + message + email. View sets order.refund_requested=True and creates a Refund record. Admin approves via bulk action in Django admin which sets refund_granted=True.

5. Forms

Form	Fields	Purpose
CheckoutForm	shipping_address, shipping_address2, shipping_country (CountryField), shipping_zip, billing_address, billing_address2, billing_country, billing_zip, same_billing_address, set_default_shipping, use_default_shipping, set_default_billing, use_default_billing, payment_option (RadioSelect)	Full checkout address collection. All address fields are required=False to support the use_default and same_billing_address shortcuts.
CouponForm	code (TextInput)	Captures a promo code string. Validated against the Coupon model in AddCouponView.
RefundForm	ref_code, message (Textarea), email (EmailField)	Customer refund request form. ref_code matched against Order.ref_code.
PaymentForm	stripeToken (hidden, from Stripe.js), save (BooleanField), use_default (BooleanField)	Carries the Stripe token from the frontend to the PaymentView POST handler.

6. Authentication — django-allauth

User authentication is fully delegated to django-allauth, mounted at /accounts/. This provides a complete auth system out of the box including registration, login, logout, email confirmation, password change, and password reset workflows. All allauth templates are customised and live in templates/account/.

Feature	Status	Notes
Registration (signup)	Complete	templates/account/signup.html
Login	Complete	templates/account/login.html
Logout	Complete	templates/account/logout.html
Email confirmation	Complete	Email templates in templates/account/email/
Password reset	Complete	Full flow: request, email, key, done pages
Password change	Complete	templates/account/password_change.html
Social login (OpenID/OAuth)	Wired up	URLs and templates present; providers not configured in settings
One-click purchasing	Custom	UserProfile.one_click_purchasing flag + Stripe Customer API

7. Django Admin

The admin interface is heavily customised for the Order and Address models, giving store managers full visibility and control over the order pipeline.

7.1 OrderAdmin

- list_display: user, ordered, being_delivered, received, refund_requested, refund_granted, shipping_address, billing_address, payment, coupon — the full order status at a glance.
- list_filter: ordered, being_delivered, received, refund_requested, refund_granted — one-click filtering by any stage.
- search_fields: user__username, ref_code — find any order by customer name or order reference.
- Bulk action: make_refund_accepted — selects multiple orders and sets refund_requested=False, refund_granted=True in one operation.

7.2 AddressAdmin

- list_display: user, street_address, apartment_address, country, zip, address_type, default.
- list_filter: default, address_type, country.
- search_fields: user, street_address, apartment_address, zip.

All other models (Item, OrderItem, Payment, Coupon, Refund, UserProfile) are registered with default admin views.

8. Frontend

8.1 Template Structure

Template	Purpose
base.html	Master layout: loads MDB CSS/JS, jQuery, Bootstrap; defines content block
navbar.html	Top navigation bar; uses cart_item_count template filter to show live cart badge
footer.html	Site footer
scripts.html	Bottom-of-page JS includes
home.html	Product listing grid — iterates over paginated Item queryset
product.html	Product detail: image, title, price, discount price, category badge, add-to-cart button
order_summary.html	Cart overview: line items, quantities, subtotals, coupon form, checkout button
order_snippet.html	Reusable cart line-item partial
checkout.html	Full checkout form: shipping address, billing address, coupon input, payment method radio
payment.html	Stripe.js payment form: card input, save card checkbox, charge button
request_refund.html	Refund request form: ref_code, message, email
account/*	All allauth auth pages: login, signup, password reset, email confirmation, etc.

8.2 Frontend Libraries

- Bootstrap 4 — responsive grid and component library.
- Material Design for Bootstrap (MDB) — enhanced Bootstrap components with Material Design styling.
- jQuery 3.3.1 — DOM manipulation and event handling.
- Stripe.js — loaded from Stripe's CDN in payment.html; tokenises card details client-side before form submission so raw card data never touches the Django server.
- DataTables — included in static assets for potential admin table enhancements.
- Custom SCSS — style.scss and component partials in static_in_env/scss/ for project-specific overrides.

8.3 cart_item_count Template Tag

A custom Django template filter registered in core/templatetags/cart_template_tags.py. It takes the current user object as input and returns the number of distinct items in the active

(ordered=False) order. The navbar includes this filter to show a live cart badge without requiring a context processor.

```
{{ user|cart_item_count }}
```

9. Implementation Approach

9.1 Prerequisites

- Python 3.7 or later
- pip and virtualenv
- A Stripe account with test API keys (for development) and live keys (for production)
- PostgreSQL (production only; SQLite is used for development)

9.2 Local Development Setup

Step	Command / Action	Notes
1. Create virtualenv	pip install virtualenv && virtualenv env	Creates isolated Python environment
2. Activate virtualenv	source env/bin/activate (Mac/Linux)	Use env\Scripts\activate on Windows
3. Install dependencies	pip install -r requirements.txt	Installs Django, Stripe, allauth, crispy-forms, decouple, Pillow, etc.
4. Configure .env	Edit .env in project root	Set SECRET_KEY, DEBUG=True, STRIPE_TEST_* keys
5. Point to dev settings	export DJANGO_SETTINGS_MODULE=djecommerce.settings.development	Or set in .env / IDE run config
6. Run migrations	python manage.py migrate	Creates all 8 core tables plus allauth, session, admin tables
7. Create superuser	python manage.py createsuperuser	Admin login for /admin/
8. Collect static (optional)	python manage.py collectstatic	Only needed if testing static serving
9. Start dev server	python manage.py runserver	App available at http://127.0.0.1:8000/

9.3 Environment Variables Reference

Variable	Dev Value	Prod Value	Purpose
SECRET_KEY	Any secret string	Long random string	Django signing key

DEBUG	True	False	Enable/disable debug mode
DB_NAME	— (SQLite used)	Your DB name	PostgreSQL database name
DB_USER	— (SQLite used)	DB user name	PostgreSQL user
DB_PASSWORD	— (SQLite used)	DB password	PostgreSQL password
DB_HOST	— (SQLite used)	localhost or host	PostgreSQL host
STRIPE_TEST_PUBLIC_KEY	pk_test_...	—	Stripe.js public key (dev)
STRIPE_TEST_SECRET_KEY	sk_test_...	—	Server-side Stripe API key (dev)
STRIPE_LIVE_PUBLIC_KEY	—	pk_live_...	Stripe.js public key (prod)
STRIPE_LIVE_SECRET_KEY	—	sk_live_...	Server-side Stripe API key (prod)

9.4 Stripe Integration Walkthrough

The payment flow uses Stripe's legacy Charges API. The sequence from button click to confirmed order is:

- 1. Browser loads payment.html — Stripe.js is loaded from Stripe's CDN.
- 2. User enters card details — Stripe.js tokenises them client-side and inserts the token into the hidden stripeToken field.
- 3. Form submits to POST /payment/stripe/ — Django never sees raw card data.
- 4. PaymentView.post() reads the token from PaymentForm, determines charge path (save / use_default / one-off).
- 5. stripe.Charge.create() is called with amount in cents, currency='usd', and either a source token or customer ID.
- 6. On success: Payment record created, Order finalised, 20-char ref_code generated.
- 7. On failure: one of six Stripe exception types is caught, user sees a specific flash message, order remains active.

9.5 Production Deployment Notes

- Switch DJANGO_SETTINGS_MODULE to djecommerce.settings.production.
- Set all .env variables for PostgreSQL and Stripe live keys.
- Update ALLOWED_HOSTS in production.py with your actual domain and IP.
- Run python manage.py collectstatic — serve static_root/ via Nginx or a CDN (e.g. AWS S3 + WhiteNoise).
- Set up a WSGI server such as Gunicorn behind Nginx: gunicorn djecommerce.wsgi:application.
- Ensure HTTPS is enforced — Stripe requires it for live key payments and allauth email confirmation links.
- Set SITE_ID=1 and configure the Sites framework domain in the Django admin (required for allauth email links).

9.6 Adding Products

Products are managed entirely through the Django admin (/admin/). To add a product: log in as superuser, go to Core > Items > Add Item, fill in title, price, optional discount price, category, label, slug (must be unique), description, and upload an image. The product immediately appears on the home page.

9.7 Extending the Application

- PayPal support: The CheckoutForm already includes PayPal as a payment_option choice ('P'). The redirect to /payment/paypal/ is wired in CheckoutView.post() but PaymentView only handles Stripe — a PayPal SDK integration would complete this.
- Search and filtering: Add a Django search bar on the HomeView using Q objects on Item.title and Item.description.
- Order history page: A ListView filtered to Order.objects.filter(user=request.user, ordered=True) gives users a purchase history.
- Email notifications: Hook into the post_save signal on Order or Payment to send transactional emails via Django's email backend.
- Product reviews / ratings: The rating.js file is already included in static assets — a Review model linking User, Item, and a score field would complete this.
- Inventory management: Add a stock field to Item and decrement it in add_to_cart; block adding out-of-stock items.

9.8 Known Limitations

- Stripe Charges API is deprecated: stripe.Charge.create() and the token-based flow are legacy. Modern Stripe recommends PaymentIntents API for SCA (Strong Customer Authentication) compliance in Europe.
- No HTTPS enforcement in code: Django's SECURE_SSL_REDIRECT and HSTS settings are not set in production.py — these should be added for production deployments.
- PayPal option is incomplete: The UI offers PayPal as a payment method but the backend only handles Stripe — selecting PayPal routes to a non-functional endpoint.
- Pillow image storage: Product images use Django's default FileSystemStorage (MEDIA_ROOT). Production deployments need cloud storage (e.g. django-storages + S3).
- No pagination on order history: There is no built-in order history view for customers.
- ref_code uniqueness: create_ref_code() generates a 20-char random string but there is no database unique constraint — an extremely rare collision is theoretically possible.

10. Summary

This Django e-commerce application is a comprehensive, production-aware reference project demonstrating real-world patterns: environment-split settings, a full payment lifecycle with Stripe, address management with default recall, coupon discounting, order status tracking, refund workflows, and a heavily customised Django admin. The single-app architecture keeps the codebase focused and easy to navigate while covering all major features needed for a functioning online store.

The primary gaps for a production-ready deployment are upgrading to Stripe's PaymentIntents API for SCA compliance, completing the PayPal integration, switching to cloud storage for media files, and adding HTTPS enforcement settings. The existing architecture is well-structured for these additions.

Feature / Module	Status	Location
Product catalogue (browse, detail page)	Complete	HomeView, ItemDetailView
Shopping cart (add / remove / adjust qty)	Complete	add_to_cart, remove_from_cart, remove_single_item_from_cart
Order summary page	Complete	OrderSummaryView, order_summary.html
Checkout (shipping + billing address)	Complete	CheckoutView, CheckoutForm
Default address recall at checkout	Complete	Address model + CheckoutView logic
Coupon code redemption	Complete	AddCouponView, Coupon model, Order.get_total()
Stripe payment (token + saved card)	Complete	PaymentView, PaymentForm, stripe SDK
One-click purchasing via Stripe Customer	Complete	UserProfile, PaymentView
Order lifecycle flags	Complete	Order model boolean fields
Refund request + admin approval	Complete	RequestRefundView, Refund model, OrderAdmin bulk action
User authentication (allauth)	Complete	django-allauth at /accounts/
Django admin customisation	Complete	core/admin.py
Environment-split settings (dev/prod)	Complete	djecommerce/settings/
PayPal payment option	Partial (UI only)	CheckoutForm — backend not implemented
Stripe PaymentIntents (SCA-compliant)	Not implemented	—
Cloud media storage	Not implemented	—
Customer order history page	Not implemented	—